

Hash table collision handling



what happens if an element does not get a valid index / place in the hash table for double hashing method?

In **double hashing**, if an element does **not find a valid index** (i.e., a free slot) during insertion into the hash table, it typically means **the table is full** or **the probe sequence fails to cover all slots**. Here's what happens and why:

Double Hashing Basics

Double hashing uses two hash functions:

- $h1(\text{key})$ — the primary hash function.
- $h2(\text{key})$ — the secondary hash function (must not be zero).

The probing sequence is:

```
ini  
  
index = (h1(key) + i * h2(key)) % table_size
```

where $i = 0, 1, 2, \dots$ until a slot is found.

! What if No Valid Index is Found?

1. If the Table is Full:

- All slots are occupied.
- No matter how many times you probe, you won't find an empty slot.
- ➤ **Insertion fails** (overflow).

2. If $h2(\text{key})$ and table_size are not coprime:

- The probe sequence **does not cover all indices** in the table.
- You may enter a **cycle** and miss some slots.
- ➤ Some free slots might exist, but are **never reached**.

✓ How to Avoid This

To ensure double hashing works properly:

- Make `table_size` a **prime number**.
- Design `h2(key)` so that it is **relatively prime** to the table size.
 - A common choice: $h2(key) = R - (key \% R)$, where R is a prime smaller than `table_size`.

🧠 Example

cpp

```
// table size = 7 (prime), R = 5
h1(key) = key % 7
h2(key) = 5 - (key % 5)

key = 10:
h1(10) = 3
h2(10) = 5 - (10 % 5) = 5 - 0 = 5

Probe sequence:
(3 + 0*5) % 7 = 3
(3 + 1*5) % 7 = 1
(3 + 2*5) % 7 = 6
...
```

It will try all slots if necessary. But if `h2` and table size are not coprime, this won't happen.

🧩 Summary

If a valid index is not found in double hashing:

- It's **either** because the table is **full**,
- **Or** the probe sequence doesn't cover all slots (due to a bad `h2`).

✓ **Always use a prime table size** and ensure `h2` is **coprime** with it to avoid this issue.



what would be the conclusion?

✓ Conclusion

In double hashing, **if an element cannot find a valid index**, it means:

- **Either** the hash table is **full** and no empty slot exists,
- **Or** the probing sequence **fails to visit all slots** due to a poorly chosen secondary hash function (h_2) that is **not relatively prime** to the table size.



To ensure every slot can be probed and avoid insertion failure:

- Use a **prime number** as the table size.
- Choose $h_2(\text{key})$ such that it is **always non-zero** and **relatively prime** to the table size.

This guarantees that the entire table is accessible during probing, preventing unnecessary insertion failures.